TECMILENIO

PROJECT PROGRESS

Nombre:

- David Moreno
- Gustavo Martinez
- Emilio Gallegos
- Diego Moya

Matrícula:

- AL03081455
- AL02962078
- AL03107568
- AL03089635

Materia: Knowledge Management Systems and Technologies

Maestro: Alfredo Paul Aguirre Torres

Tarea: Project Progress

Fecha: 29/10/2026

Nexo IT Services

Project progress

Nexo IT Services
Sistemas y tecnologías de administración del conocimiento

Organization and problem

Nexo IT Services is a fictional managed IT support provider for small and medium-sized organizations. Its core work includes resolving employee access incidents, maintaining backup and recovery procedures, coordinating technical changes, responding to security alerts and training support analysts. All company details and documents in this project are simulated.

Knowledge is distributed across incident records, approved procedures, policies, FAQs and learning notes. An experienced analyst may recognize a stale credential immediately, while a new analyst must search several records to find the same recovery steps. This project captures that experience as explicit, classified and connected documents that can be found using a natural language query.

Team or role	Core activity	Knowledge produced or used
Service desk	Diagnose tickets and document validated resolutions	Incident records, FAQs, escalation notes
Network and identity team	Maintain remote access and authentication	Access policies, recovery procedures
Operations and security	Maintain recovery, changes and response controls	Backup policies, change plans, response guidance
Knowledge lead and document owners	Review, publish, train and revise articles	Training notes, metadata, approved versions

Document collection and intended users

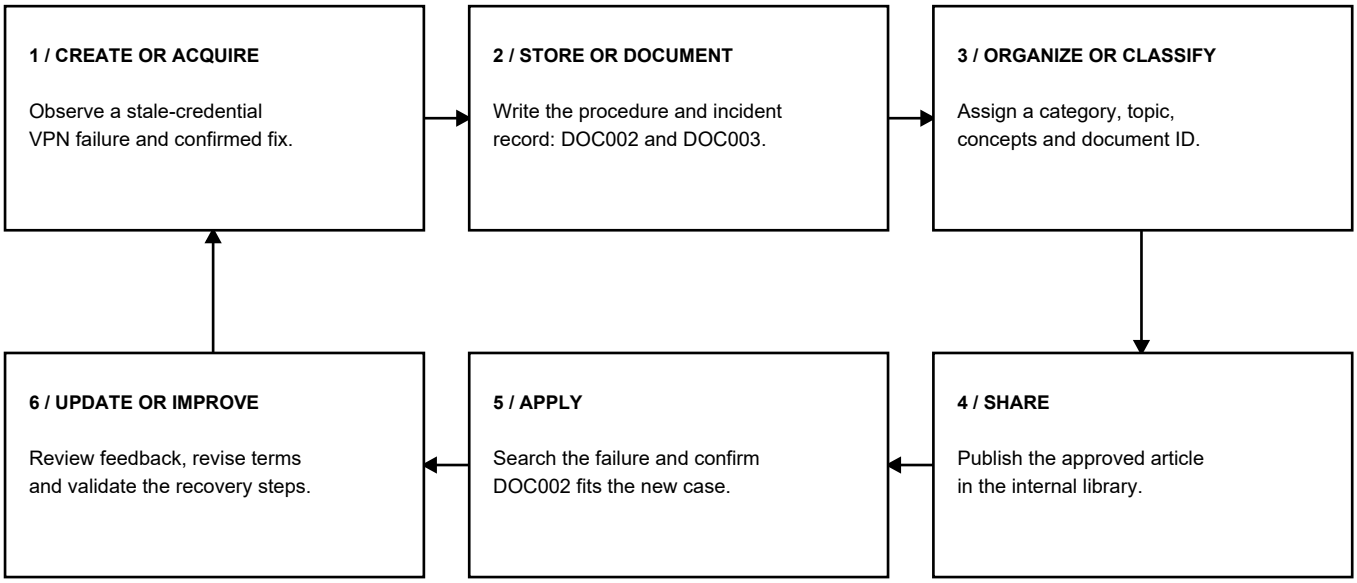
The repository contains 30 original English documents: 11 Policy/Process, 9 Support/Incidents and 10 Knowledge/Training. Each CSV record has an ID, title, text, category, topic, concepts, owner, version, review date and source. Service desk analysts are the primary users; document owners review content and classification suggestions.

Progress objective

Deliver a six-stage knowledge lifecycle map, a conceptual ontology and an executable Python prototype that loads the corpus, classifies new text and ranks documents for two search scenarios. The initial interface runs the exported Python model in the browser. This phase establishes a reproducible baseline for the later course activities.

Knowledge lifecycle

A confirmed VPN recovery is the running example. Arrows connect six stages and return feedback to the next creation cycle.



Stage	Responsible role	Why this stage matters
Create or acquire	Service desk	Capture the observed error, diagnosis and verified fix before the experience is lost.
Store or document	Knowledge lead	Turn the explanation into a durable procedure and incident record with stable IDs.
Organize or classify	Document owner	Assign category, topic and concepts so the record can be located and connected.
Share	Knowledge lead	Make the reviewed version available to peers through the internal library.
Apply	Service desk	Check whether DOC002 fits the new failure and follow its approved recovery steps.
Update or improve	Owner and network team	Use failed searches, changed clients and incident feedback to revise and validate content.

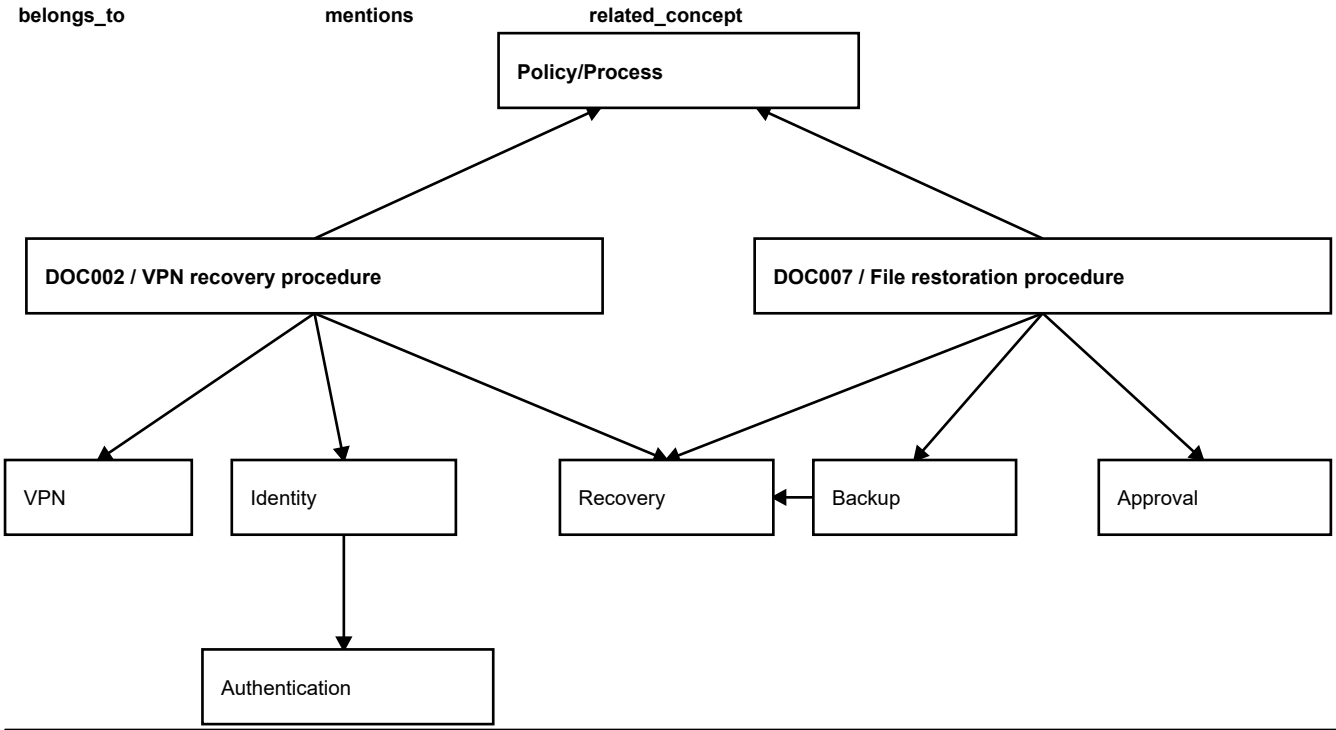
Practical application

The service desk externalizes tacit diagnosis into DOC003 and the approved procedure DOC002. Combining that incident with policy and FAQ records makes the experience reusable across teams. A supervised analyst internalizes the knowledge by applying the procedure and documenting the result. A failed search for "corporate tunnel" would trigger vocabulary review, because the baseline does not automatically equate that phrase with "VPN".

This map is a proposed operating process for the fictional organization. The prototype implements organization and retrieval; it does not implement an approval workflow, access control or automatic document revision.

Organizational ontology

The graph connects document types and domain concepts. This readable subset contains two real corpus records; the complete baseline is exported as directed triples.



Entity or predicate	Definition and example
Document	An identifiable knowledge record, such as DOC002 or DOC007.
Category	One of the three predefined document classes.
Concept	A controlled domain term, such as VPN, Identity, Backup or Recovery.
Document belongs_to Category	DOC002 belongs_to Policy/Process; exactly one category per document.
Document mentions Concept	DOC007 mentions Backup, Recovery and Approval. These are curated metadata assertions.
Concept related_concept Concept	Identity relates to Authentication; Backup relates to Recovery. The stored triples have an explicit direction.

Complete baseline and integrity checks

The full graph contains 60 nodes: 30 documents, 3 categories and 27 concepts. Its 126 directed triples comprise 30 category memberships, 90 document-concept mentions and 6 concept relations. All node IDs are unique, every edge endpoint exists, every document has exactly one category, and every document has concept metadata. All four executable checks passed.

Recovery connects the VPN procedure and the file restoration procedure, exposing a reusable concept across two operational topics. A shared concept does not imply that the same procedure applies to both failures. Category membership comes from the reviewed corpus labels; a new-text prediction does not silently alter this graph.

Evidence: evidencias/ontology.json and ontology_triples.csv. This is a lightweight conceptual ontology and triple export.

Classification and search

1. Load and validate the corpus

recommender.py reads corpus.csv with UTF-8 encoding and verifies the required ID, title and text fields, nonempty records, unique identifiers and a sufficient collection size. progress.py uses the additional category and concept metadata to train the classifier and generate the graph. The included corpus is the complete original 30-document collection.

2. Represent documents with TF-IDF

The shared preprocessing function normalizes case and accents and replaces special characters with spaces. Tokenization retains alphanumeric tokens of at least two characters and removes a fixed English stopword list. The vectorizer uses unigrams and bigrams, sublinear term frequency, smoothed inverse document frequency and L2 normalization. Fitting the search collection produces 2,317 features.

For a term t, sublinear TF is 1 + log(count) when present; smoothed IDF is log((1 + N)/(1 + df(t))) + 1. The weighted vector is normalized to unit length. A query passes through the same fitted vocabulary; unseen terms contribute no feature.

3. Learn predefined document categories

classifier.py fits a supervised logistic regression model to TF-IDF text features using the reviewed labels Policy/Process, Support/Incidents and Knowledge/Training. Parameters are C=4, max_iter=1500 and random_state=42. The demonstration classifier is refitted on all 30 records. It predicts a category for new input and displays all three uncalibrated probabilities for human review.

An independent stratified 70/30 split trains a separate classifier on 21 records and evaluates 9 held-out records. Its vocabulary is fitted only on the training text. The held-out model is used to assess limitations; it is distinct from the full-corpus browser demonstration.

4. Rank documents for a natural language query

The search transforms the query using the fitted search vectorizer, computes cosine similarity against every document and sorts results by descending score, using document ID to break ties. The interface displays the five highest-ranked IDs, titles and scores. Empty queries are rejected. An out-of-vocabulary query produces zero similarities and must be rephrased.

This implements the course's basic "semantic search" pathway with TF-IDF and cosine similarity. It is a lexical baseline: it compares weighted shared words and phrases, without neural semantic embeddings or language-model generation.

5. Reproduce the browser demonstration

progress.py exports document vectors, vocabulary, IDF weights and classifier coefficients into progress-model.json. progress-engine.js performs the same normalization, vector transformation, dot products and multiclass probability calculation. Automated parity checks matched both searches and all three classification cases to Python within 10^-12.

Runtime: Python 3.12.14, NumPy 2.5.4 and scikit-learn 1.9.1. Six functional tests passed. Files: progress.py, recommender.py, classifier.py, test_progress.py and test_parity.mjs. No external API key is required.

Classification results

1. Policy example

Every production change must include manager approval, a validation plan and a rollback procedure.

Predicted category	Model output	Other class outputs
Policy/Process	0.439906	Knowledge/Training: 0.319246; Support/Incidents: 0.240848

2. Incident example

An employee reported a failed VPN connection after a password change. The analyst cleared cached credentials and restored access.

Predicted category	Model output	Other class outputs
Support/Incidents	0.389764	Knowledge/Training: 0.280080; Policy/Process: 0.330156

3. Training example

During the training workshop, participants practice restoring a sample folder and compare the backup manifest in a learning exercise.

Predicted category	Model output	Other class outputs
Knowledge/Training	0.542299	Policy/Process: 0.253221; Support/Incidents: 0.204480

The policy input emphasizes approval and rollback, the incident input describes a failure and resolution, and the training input describes a practice exercise. These curated examples demonstrate the prediction path. Their agreement with the intended categories is not evidence of reliable performance on unseen operational material.

Independent held-out evaluation

Held-out ID	Reviewed category	Prediction
DOC024	Knowledge/Training	Policy/Process
DOC015	Support/Incidents	Policy/Process
DOC026	Support/Incidents	Policy/Process
DOC016	Policy/Process	Support/Incidents
DOC010	Knowledge/Training	Knowledge/Training
DOC006	Policy/Process	Knowledge/Training
DOC008	Support/Incidents	Policy/Process
DOC014	Knowledge/Training	Policy/Process
DOC017	Policy/Process	Policy/Process

The nine-record holdout contains three documents per class. Accuracy is 0.222; macro precision is 0.222, macro recall is 0.222 and macro F1 is 0.207. Only two held-out records were classified correctly. The small corpus and overlapping operational vocabulary make this an unreliable automatic routing model.

The intended use is a category suggestion reviewed by a person. Improvements should add independently labeled examples, clearer category guidelines and a separate validation set. Probability outputs are uncalibrated and do not express a guaranteed correctness rate. Raw inputs, probabilities, train/test IDs, per-class metrics and confusion matrix are included in classification.json and holdout.json.

Search results

Search 1

Query: Restore deleted customer files after a failed backup

Rank	ID / title	Cosine
1	DOC007 / File restoration operating procedure	0.321625
2	DOC009 / Deleted files recovery FAQ	0.218788
3	DOC010 / Quarterly restoration exercise	0.190034
4	DOC006 / Backup retention policy	0.125877
5	DOC008 / Backup job failed because storage was full	0.096592

DOC007 ranks first because its restoration instructions share the query's file-recovery vocabulary. DOC009 supplies a recovery FAQ, while DOC010 is a simulated exercise. DOC006 provides retention context and DOC008 explains a failed backup incident. The analyst should confirm that a usable recovery point exists and follow the governing procedure; a training example is not approval to restore customer data.

Search 2

Query: VPN connection fails after password change

Rank	ID / title	Cosine
1	DOC002 / VPN connection recovery procedure	0.334634
2	DOC003 / VPN failure following credential rotation	0.214481
3	DOC005 / Secure remote working workshop	0.169119
4	DOC001 / Remote access policy	0.107147
5	DOC004 / Remote connection troubleshooting FAQ	0.102538

DOC002 directly addresses a VPN failure after a password change, and DOC003 documents the same incident pattern. DOC001 supplies access-policy context and DOC004 a troubleshooting FAQ. DOC005 ranks third because it repeats query terms even though it explicitly provides no recovery procedure. Its lexical overlap illustrates how similarity can diverge from operational usefulness.

What the ranking reveals

Read the source before applying a result. Scores measure cosine similarity, not relevance, factual correctness or permission to perform an action. The relevant alternate-language record DOC026 uses "corporate tunnel", "secret" and "stale token" and is absent from the VPN top five. A future iteration should test controlled synonyms or semantic embeddings on an independently reviewed query set.

Evidence: searches.json stores the full retrieved records and scores; execution.log records both executions. The next page shows the same results in the working interface. The corpus, fitted vocabulary and ranking function are unchanged across these two cases.

Prototype screenshots

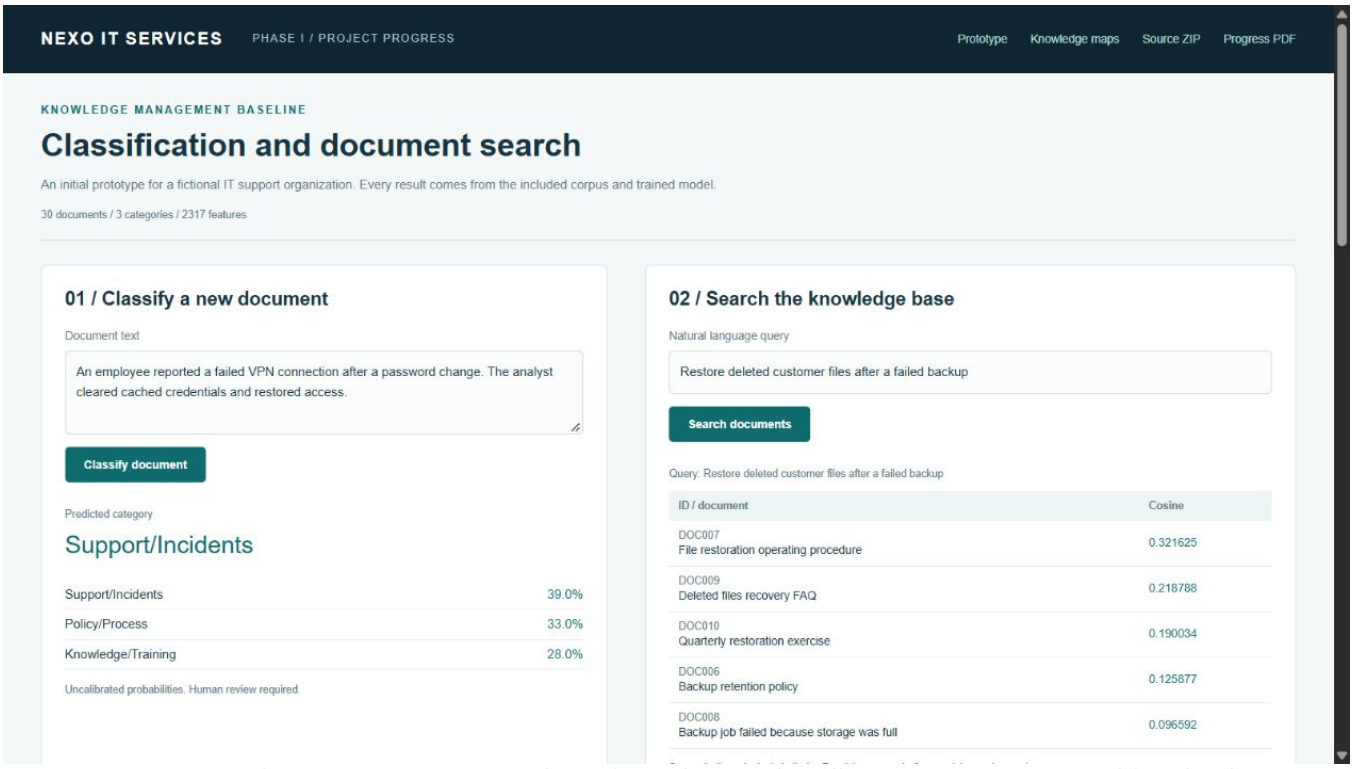


Figure 1. Local Phase I interface: the incident input is classified as Support/Incidents (39.0%). The backup query returns DOC007 first, followed by DOC009, DOC010, DOC006 and DOC008.

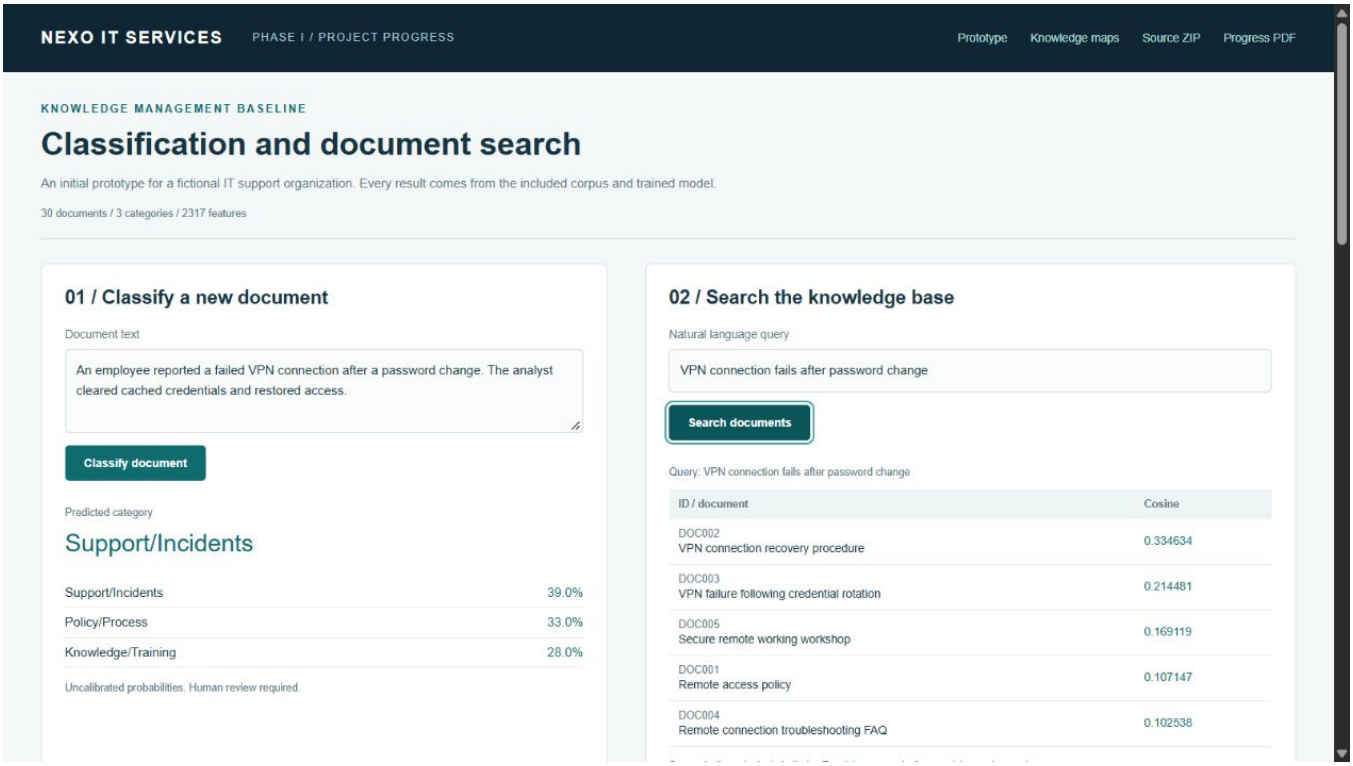


Figure 2. The query changes to the VPN failure scenario. The interface returns DOC002, DOC003, DOC005, DOC001 and DOC004 with the same six-decimal scores as the Python evidence.

Applied exercises and source code

The following exercises apply the six course topics to the Nexo prototype. Each record explains what was done and identifies the files where the result can be checked.

Topic reviewed	Applied practice and evidence	Contribution
1. Introduction to Knowledge Management	01_knowledge_inventory.md: tacit/explicit inventory, DIKW and SECI mapping	Explains why an incident becomes explicit reusable knowledge and a six-stage cycle.
2. AI as an Enabler of Knowledge Management	02_ai_classification.md and classification.json: three new inputs	Uses supervised AI to assist document organization while preserving human review.
3. Knowledge representation in AI	03_ontology.md and ontology_triples.csv: typed entities and predicates	Makes knowledge relationships explicit and testable.
4. Knowledge Graphs and Their Organizational Application	04_graph_mapping.md: trace two procedures through shared Recovery	Connects documents, types and concepts across operational topics.
5. Automated document classification with AI	05_classification_preprocessing.md and holdout.json	Applies preprocessing, TF-IDF, supervised training and a separate weak-result evaluation.
6. Information Retrieval and Semantic Search	06_information_retrieval.md and searches.json	Demonstrates query ranking and explains vocabulary and relevance limitations.

Run and inspect the repository

Extract Progress_Source.zip and install requirements.txt in a Python virtual environment. Run **python progress.py build** to regenerate JSON, graph triples and practice records; run **python -m unittest test_progress.py -v** for the six checks. With Node.js available, run **node test_parity.mjs** to compare browser calculations with the Python evidence.

To open the local interface, run **python -m http.server 8766 --bind 127.0.0.1 --directory site**, then open http://127.0.0.1:8766/progress.html. The README provides full environment setup, CLI examples, limitations and dependency versions. Source ZIP includes the corpus, implementation, prebuilt interface, execution records, maps and screenshots.

Public prototype: Nexo IT Phase I
Source files: Progress_Source.zip
Portable Git repository: Progress_Source.bundle
Progress report: Project_Progress_Report.pdf

References and scope

Tecmilenio. Sistemas y tecnologías de administración del conocimiento, Topics 1-6 and project progress instructions, course 250962. Project progress assignment. Course pages reviewed 27 October 2026; course access may require sign-in.

scikit-learn. TfidfVectorizer; LogisticRegression; classification_report. Official API documentation.

The baseline is sufficient to demonstrate classification and search execution, but its poor held-out classification results prevent operational automation. Later course deliveries expand recommendation, evaluation and conversation features. This Phase I package remains independently reproducible.